

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN
VIỆN TRÍ TUỆ NHÂN TẠO



KỸ THUẬT VÀ CÔNG NGHỆ
DỮ LIỆU LỚN
CHO TRÍ TUỆ NHÂN TẠO

BÁO CÁO BÀI TẬP LỚN CUỐI KỲ

Triển khai hệ thống phát hiện giao dịch gian lận
sử dụng phân tích dữ liệu lớn và học máy

Sinh viên: Vũ Hoàng Diệu Linh
Mã sinh viên: 24022387
Lớp học phần: AIT3229_1

Hà Nội, 06/2026

LỜI NÓI ĐẦU

- Source code: [Github](#)
- Dataset: [Kaggle](#)

Lời đầu tiên, em xin gửi lời cảm ơn chân thành đến thầy Trần Hồng Việt và cô Ngô Minh Hương, giảng dạy môn Kỹ thuật và công nghệ dữ liệu lớn cho Trí tuệ nhân tạo, đã tận tình hướng dẫn, truyền đạt kiến thức và tạo điều kiện để em và các bạn trong lớp có cơ hội được vận dụng các kiến thức đã học vào bài tập lớn lần này.

Trong bối cảnh các giao dịch tài chính điện tử ngày càng phát triển, nguy cơ gian lận trong thanh toán số, thẻ ngân hàng và thương mại điện tử cũng trở thành một vấn đề đáng quan tâm. Với số lượng giao dịch lớn và liên tục phát sinh, việc phát hiện gian lận bằng phương pháp thủ công không còn phù hợp. Vì vậy, trong bài tập lớn này, em lựa chọn đề tài "**Fraud Detection System using Big Data Analytics and Machine Learning**" hay triển khai hệ thống phát hiện giao dịch gian lận bằng cách kết hợp phân tích dữ liệu lớn và học máy. Cụ thể, em tiến hành phân tích dữ liệu, tiền xử lý và xây dựng đặc trưng, sử dụng Apache Spark cho các tác vụ xử lý dữ liệu lớn, đồng thời huấn luyện, đánh giá và so sánh kết quả giữa các mô hình như Logistic Regression, Random Forest và HistGradientBoosting.

Em đã cố gắng hoàn thiện bài tập trong khả năng của mình với kỳ vọng có thể củng cố kiến thức và kinh nghiệm về các kỹ thuật công nghệ về dữ liệu lớn, là nền tảng định hướng em có thể phát triển thành các dự án thực có ý nghĩa đóng góp cho xã hội ngày càng phát triển này. Dù vậy, báo cáo khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được sự góp ý của thầy cô để có thể tiếp tục hoàn thiện kiến thức, cải thiện phương pháp thực hiện và phát triển đề tài tốt hơn trong tương lai. Một lần nữa, em xin trân trọng cảm ơn thầy cô và các bạn đã đồng hành và hỗ trợ trong suốt quá trình học tập, tìm hiểu, nghiên cứu và hoàn thành bài tập này nói riêng và các dự án đã đang và sẽ phát triển nói chung.

Trân trọng,
Diệu Linh.

Mục lục

1	Giới thiệu	3
1.1	Đặt vấn đề	3
1.2	Mục tiêu	3
1.3	Phạm vi thực hiện	4
2	Mô tả, phân tích, tiền xử lý dữ liệu và xây dựng đặc trưng	5
2.1	Mô tả bộ dữ liệu	5
2.2	Đặc trưng dữ liệu lớn của bài toán	5
2.3	Phân tích khám phá dữ liệu	6
2.4	Tiền xử lý dữ liệu	8
2.5	Xây dựng đặc trưng	9
3	Xử lý dữ liệu lớn với Apache Spark	10
3.1	Vai trò của Apache Spark trong đề tài	10
3.2	Quy trình xử lý dữ liệu bằng Spark	10
3.3	Các phân tích thực hiện bằng Spark	10
3.4	Ý nghĩa của Spark đối với quá trình xây dựng mô hình	11
3.5	So sánh Pandas và Spark	12
4	Xây dựng mô hình phát hiện giao dịch gian lận	13
4.1	Phát biểu bài toán	13
4.2	Xử lý mất cân bằng dữ liệu	13
4.3	Các mô hình sử dụng	14
4.4	Quy trình huấn luyện mô hình	15
4.5	Điều chỉnh ngưỡng dự đoán (Threshold Tuning)	16
5	Kết quả thực nghiệm và đánh giá	17
5.1	Các độ đo đánh giá	17
5.2	Kết quả so sánh các mô hình	17
5.3	Phân tích mô hình tốt nhất	18
5.4	Trực quan hóa kết quả đánh giá	19
5.5	Nhận xét	21
6	Kết luận và hướng phát triển	22
6.1	Kết luận	22
6.2	Hạn chế của đề tài	22
6.3	Hướng phát triển	22

1 Giới thiệu

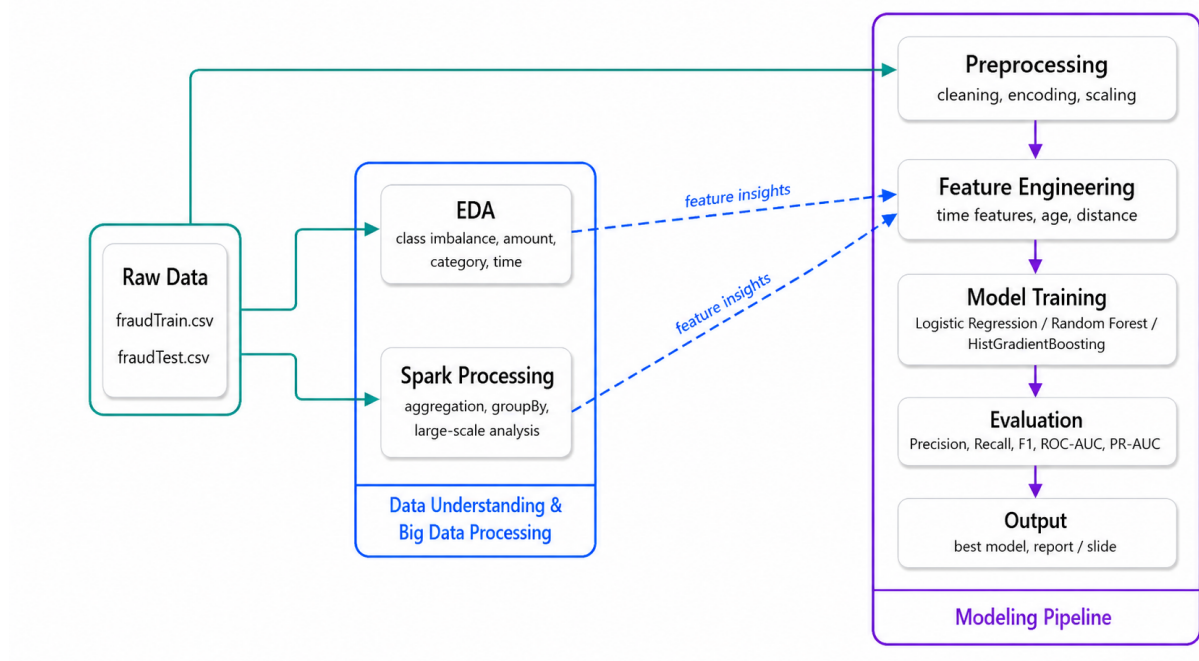
1.1 Đặt vấn đề

Trong bối cảnh các giao dịch tài chính điện tử ngày càng phổ biến thông qua ngân hàng số, thẻ thanh toán, ví điện tử và thương mại điện tử, vấn đề gian lận giao dịch trở thành một rủi ro đáng quan tâm. Các hành vi gian lận không chỉ gây thiệt hại tài chính mà còn ảnh hưởng đến độ tin cậy của hệ thống thanh toán.

Với số lượng giao dịch lớn và liên tục phát sinh, việc kiểm tra thủ công từng giao dịch là không khả thi. Vì vậy, cần xây dựng các hệ thống tự động có khả năng phân tích dữ liệu lịch sử để phát hiện các giao dịch có nguy cơ gian lận. Tuy nhiên, bài toán này có độ khó cao do số lượng giao dịch gian lận thường chiếm tỷ lệ rất nhỏ so với giao dịch hợp lệ, dẫn đến hiện tượng mất cân bằng dữ liệu. Do đó, việc xây dựng mô hình cần đi kèm với cách xử lý dữ liệu phù hợp và sử dụng các độ đo đánh giá như Precision, Recall, F1-score và PR-AUC thay vì chỉ dựa vào Accuracy.

1.2 Mục tiêu

Đề tài hướng tới việc xây dựng một pipeline phát hiện giao dịch gian lận dựa trên phân tích dữ liệu lớn và học máy như hình 1 dưới đây.



Hình 1: Kiến trúc tổng thể của hệ thống phát hiện giao dịch gian lận

Cụ thể, đề tài thực hiện các mục tiêu sau:

- Tìm hiểu đặc điểm của bộ dữ liệu giao dịch tài chính;

- Phân tích sự mất cân bằng của lớp giao dịch gian lận;
- Tiền xử lý dữ liệu và xây dựng các đặc trưng phục vụ mô hình học máy;
- Sử dụng Apache Spark để tổng hợp và phân tích dữ liệu lớn;
- Huấn luyện, đánh giá và so sánh các mô hình Logistic Regression, Random Forest và HistGradientBoosting.

1.3 Phạm vi thực hiện

Trong phạm vi bài tập lớn này, em sử dụng bộ dữ liệu [Credit Card Transactions Fraud Detection Dataset](#) trên Kaggle. Đề tài tập trung vào xử lý dữ liệu theo lô trên dữ liệu có sẵn, chưa triển khai hệ thống phát hiện gian lận theo thời gian thực. Apache Spark được sử dụng cho các tác vụ xử lý và tổng hợp dữ liệu lớn, trong khi các mô hình học máy được huấn luyện và đánh giá bằng thư viện scikit-learn. Kết quả mô hình được so sánh dựa trên các độ đo phù hợp với dữ liệu mất cân bằng, đặc biệt là Recall, F1-score và PR-AUC.

Toàn bộ mã nguồn được lưu trữ tại: [GitHub](#).

2 Mô tả, phân tích, tiền xử lý dữ liệu và xây dựng đặc trưng

2.1 Mô tả bộ dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là *Credit Card Transactions Fraud Detection Dataset*, gồm các giao dịch thẻ với nhiều thông tin khác nhau như thời gian giao dịch, số tiền, loại giao dịch, thông tin merchant, thông tin khách hàng và vị trí địa lý. Dữ liệu gồm hai file chính:

- `fraudTrain.csv`: tập dữ liệu dùng cho huấn luyện;
- `fraudTest.csv`: tập dữ liệu dùng cho kiểm thử.

Biến mục tiêu của bài toán là `is_fraud`. Trong đó, giá trị 0 biểu diễn giao dịch hợp lệ và giá trị 1 biểu diễn giao dịch gian lận.

Bảng 1: Tổng quan bộ dữ liệu

Tập dữ liệu	Số giao dịch	Mục đích sử dụng
<code>fraudTrain.csv</code>	1,296,675	Huấn luyện mô hình
<code>fraudTest.csv</code>	555,719	Kiểm thử mô hình

Trong tập huấn luyện, số lượng giao dịch gian lận là 7,506 trên tổng số 1,296,675 giao dịch, tương ứng với tỷ lệ khoảng 0.5789%. Điều này cho thấy dữ liệu có mức độ mất cân bằng rất cao, khi số lượng giao dịch gian lận nhỏ hơn rất nhiều so với giao dịch hợp lệ.

Các thuộc tính trong dữ liệu có thể chia thành một số nhóm chính:

- **Thông tin giao dịch:** `trans_date_trans_time`, `amt`, `category`, `merchant`;
- **Thông tin khách hàng:** `gender`, `dob`, `job`, `city`, `state`;
- **Thông tin vị trí:** `lat`, `long`, `merch_lat`, `merch_long`;
- **Thông tin định danh:** `cc_num`, `first`, `last`, `street`, `trans_num`;
- **Biến mục tiêu:** `is_fraud`.

2.2 Đặc trưng dữ liệu lớn của bài toán

Bộ dữ liệu giao dịch tài chính có nhiều đặc trưng phù hợp với bài toán dữ liệu lớn. Có thể phân tích theo mô hình 5V như sau:

- **Volume:** dữ liệu gồm hơn 1.8 triệu giao dịch trên cả tập huấn luyện và kiểm thử, đủ lớn để cần một quy trình xử lý có tổ chức;
- **Velocity:** trong thực tế, giao dịch tài chính phát sinh liên tục và cần được phát hiện rủi ro trong thời gian ngắn;

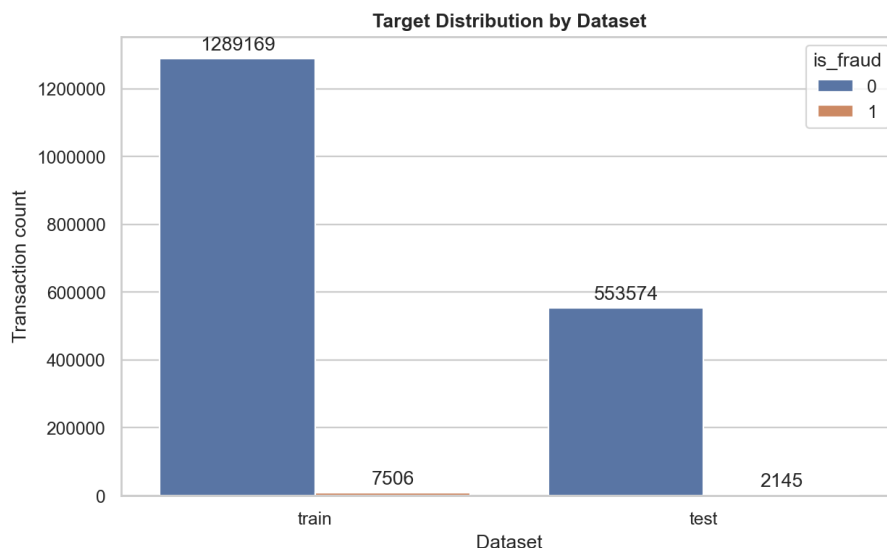
- **Variety:** dữ liệu gồm nhiều kiểu thuộc tính như số tiền, thời gian, loại giao dịch, thông tin khách hàng, merchant và vị trí địa lý;
- **Veracity:** dữ liệu cần được kiểm tra, làm sạch, xử lý kiểu dữ liệu và loại bỏ các thuộc tính định danh trực tiếp trước khi đưa vào mô hình;
- **Value:** nếu được khai thác hiệu quả, dữ liệu có thể hỗ trợ phát hiện giao dịch gian lận, giảm thiệt hại và nâng cao độ an toàn của hệ thống thanh toán.

Do đó, đề tài không chỉ là một bài toán huấn luyện mô hình học máy đơn thuần, mà còn cần quan tâm đến quy trình phân tích, xử lý dữ liệu lớn và đánh giá mô hình phù hợp với đặc điểm mất cân bằng của dữ liệu.

2.3 Phân tích khám phá dữ liệu

Phân tích khám phá dữ liệu được thực hiện nhằm hiểu rõ đặc điểm của bộ dữ liệu trước khi huấn luyện mô hình. Các nội dung phân tích chính bao gồm phân bố biến mục tiêu, phân bố số tiền giao dịch, tỷ lệ gian lận theo loại giao dịch, merchant, thời gian giao dịch và một số thuộc tính liên quan khác.

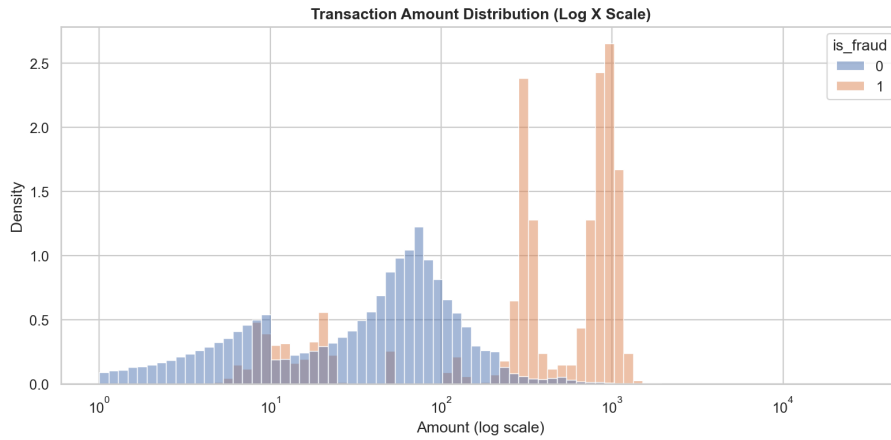
Trước hết, phân bố của biến mục tiêu `is_fraud` cho thấy dữ liệu bị mất cân bằng rất mạnh. Phần lớn giao dịch là hợp lệ, trong khi giao dịch gian lận chỉ chiếm tỷ lệ rất nhỏ. Đây là đặc điểm quan trọng ảnh hưởng trực tiếp đến cách huấn luyện và đánh giá mô hình. Nếu chỉ sử dụng Accuracy, mô hình có thể đạt điểm rất cao bằng cách dự đoán hầu hết giao dịch là hợp lệ, nhưng lại không phát hiện tốt các giao dịch gian lận.



Hình 2: Phân bố biến mục tiêu `is_fraud`

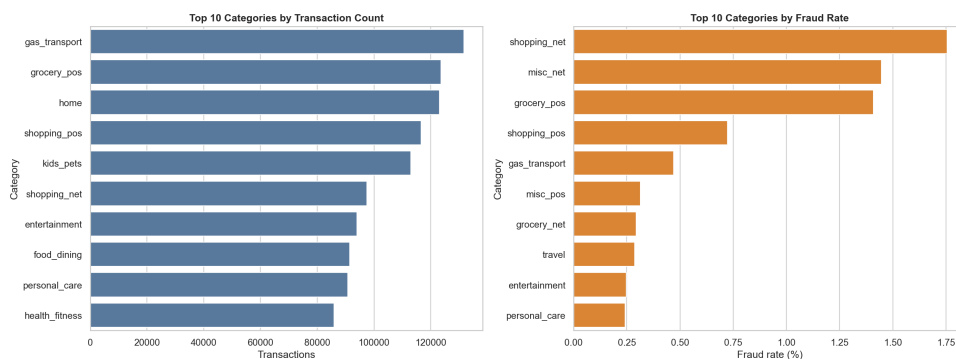
Tiếp theo, phân tích phân bố số tiền giao dịch cho thấy giá trị giao dịch có xu hướng lệch phải. Phần lớn giao dịch có giá trị nhỏ hoặc trung bình, trong khi một số ít giao dịch

có giá trị rất lớn. Do đó, biểu đồ trên thang log được sử dụng để quan sát rõ hơn phân bố của biến `amt`.



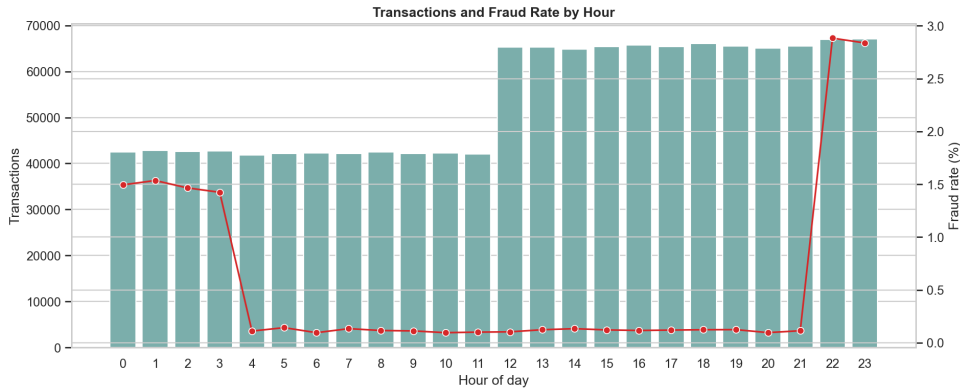
Hình 3: Phân bố số tiền giao dịch trên thang log

Ngoài ra, tỷ lệ gian lận cũng được phân tích theo loại giao dịch. Kết quả cho thấy tỷ lệ gian lận có sự khác biệt giữa các nhóm giao dịch. Một số loại giao dịch có tỷ lệ gian lận cao hơn các nhóm còn lại, cho thấy thuộc tính `category` có thể mang thông tin hữu ích cho mô hình phát hiện gian lận.



Hình 4: Số lượng giao dịch và tỷ lệ gian lận theo loại giao dịch

Phân tích theo thời gian cũng cho thấy hành vi gian lận có thể thay đổi theo giờ trong ngày. Vì vậy, các đặc trưng thời gian như giờ giao dịch, ngày trong tuần và tháng giao dịch được xem xét trong bước xây dựng đặc trưng.



Hình 5: Tỷ lệ gian lận theo giờ giao dịch

Tóm lại, quá trình phân tích khám phá dữ liệu cho thấy bài toán phát hiện gian lận có ba đặc điểm quan trọng: dữ liệu mất cân bằng mạnh, số tiền giao dịch có phân bố lệch, và tỷ lệ gian lận thay đổi theo nhiều yếu tố như loại giao dịch, merchant, thời gian và vị trí. Những kết quả này là cơ sở để lựa chọn các bước tiền xử lý và xây dựng đặc trưng phù hợp.

2.4 Tiền xử lý dữ liệu

Dữ liệu thô ban đầu không thể đưa trực tiếp vào mô hình học máy do có nhiều thuộc tính dạng chuỗi, thuộc tính thời gian, thông tin định danh và các cột có thang đo khác nhau. Vì vậy, bước tiền xử lý được thực hiện nhằm biến đổi dữ liệu về dạng phù hợp cho quá trình huấn luyện mô hình.

Trước hết, cột thời gian `trans_date_trans_time` được chuyển sang kiểu dữ liệu thời gian để có thể trích xuất các đặc trưng liên quan. Từ cột này, một số đặc trưng mới được tạo ra như giờ giao dịch, ngày trong tuần, tháng giao dịch và cờ đánh dấu cuối tuần.

Bên cạnh đó, cột `dob` được sử dụng để tính tuổi của khách hàng tại thời điểm giao dịch. Tuổi khách hàng có thể phản ánh sự khác biệt trong hành vi giao dịch giữa các nhóm người dùng khác nhau.

Các cột vị trí địa lý gồm `lat`, `long`, `merch_lat` và `merch_long` được sử dụng để tính khoảng cách giữa vị trí khách hàng và vị trí merchant. Khoảng cách bất thường giữa khách hàng và merchant có thể là một dấu hiệu hữu ích trong việc phát hiện giao dịch gian lận.

Ngoài ra, một số cột định danh trực tiếp được loại bỏ để giảm nguy cơ mô hình học thuộc dữ liệu thay vì học các quy luật tổng quát. Các cột này bao gồm `cc_num`, `first`, `last`, `street`, `trans_num`, `trans_date_trans_time` và `dob`. Việc loại bỏ các thuộc tính định danh cũng giúp mô hình phù hợp hơn khi áp dụng cho dữ liệu mới.

2.5 Xây dựng đặc trưng

Sau bước tiền xử lý cơ bản, dữ liệu được xây dựng thêm các đặc trưng mới nhằm tăng khả năng phân biệt giữa giao dịch hợp lệ và giao dịch gian lận. Các nhóm đặc trưng chính được sử dụng gồm đặc trưng thời gian, đặc trưng tuổi khách hàng, đặc trưng khoảng cách địa lý và đặc trưng mã hóa từ biến phân loại.

Các đặc trưng thời gian được tạo ra bao gồm:

- `transaction_hour`: giờ xảy ra giao dịch;
- `transaction_dayofweek`: ngày trong tuần;
- `transaction_month`: tháng xảy ra giao dịch;
- `weekend_flag`: đánh dấu giao dịch xảy ra vào cuối tuần hay không.

Đặc trưng `customer_age` được tính từ ngày sinh của khách hàng và thời điểm giao dịch. Đặc trưng `customer_merchant_distance_km` được tính từ tọa độ khách hàng và tọa độ merchant, nhằm biểu diễn khoảng cách địa lý giữa hai vị trí.

Đối với các biến phân loại, đề tài sử dụng hai phương pháp mã hóa. Các cột có số lượng giá trị không quá lớn như `category`, `gender` và `state` được mã hóa bằng one-hot encoding. Các cột có số lượng giá trị lớn hơn như `merchant`, `job` và `city` được mã hóa bằng frequency encoding, tức là thay mỗi giá trị bằng tần suất xuất hiện của giá trị đó trong tập huấn luyện.

Các đặc trưng số như `amt`, `city_pop`, `lat`, `long`, `merch_lat`, `merch_long`, `unix_time`, `customer_age` và `customer_merchant_distance_km` được chuẩn hóa để đưa về cùng thang đo. Việc chuẩn hóa giúp các mô hình học máy hoạt động ổn định hơn, đặc biệt đối với các mô hình nhạy cảm với thang đo của dữ liệu.

Sau quá trình tiền xử lý và xây dựng đặc trưng, dữ liệu thu được có 85 cột, trong đó gồm 84 đặc trưng đầu vào và 1 biến mục tiêu là `is_fraud`. Cụ thể, tập huấn luyện có 1,296,675 dòng và tập kiểm thử có 555,719 dòng.

Bảng 2: Kết quả sau tiền xử lý và xây dựng đặc trưng

Tập dữ liệu	Số dòng	Số cột sau xử lý
Train	1,296,675	85
Test	555,719	85

Các bước mã hóa và chuẩn hóa được xây dựng dựa trên tập huấn luyện, sau đó áp dụng lại cho tập kiểm thử. Cách làm này giúp hạn chế hiện tượng rò rỉ dữ liệu, tránh việc thông tin từ tập kiểm thử bị sử dụng trong quá trình huấn luyện mô hình.

3 Xử lý dữ liệu lớn với Apache Spark

3.1 Vai trò của Apache Spark trong đề tài

Trong đề tài này, Apache Spark được sử dụng như một công cụ hỗ trợ xử lý và phân tích dữ liệu lớn. Với bộ dữ liệu giao dịch có hơn 1.8 triệu bản ghi, việc sử dụng Spark giúp thể hiện khả năng xử lý dữ liệu theo hướng song song, có tổ chức và có thể mở rộng khi kích thước dữ liệu tăng lên.

Cần lưu ý rằng Spark không được sử dụng để huấn luyện mô hình chính trong đề tài. Thay vào đó, Spark được dùng để đọc dữ liệu giao dịch, thực hiện các phép tổng hợp và phân tích theo nhiều chiều khác nhau như loại giao dịch, bang, merchant, thời gian giao dịch và số tiền giao dịch. Các kết quả này giúp hiểu rõ hơn đặc điểm của dữ liệu và hỗ trợ quá trình phân tích, lựa chọn đặc trưng cho mô hình học máy.

Do đó, trong pipeline tổng thể, Spark Processing là một nhánh xử lý dữ liệu lớn song song với nhánh tiền xử lý và xây dựng mô hình. Cả hai nhánh đều xuất phát từ dữ liệu ban đầu nhưng phục vụ các mục đích khác nhau. Spark tập trung vào phân tích và tổng hợp dữ liệu lớn, trong khi Preprocessing và Feature Engineering tập trung vào việc tạo dữ liệu đầu vào phù hợp cho mô hình học máy.

3.2 Quy trình xử lý dữ liệu bằng Spark

Quy trình xử lý dữ liệu bằng Spark bắt đầu từ việc đọc dữ liệu giao dịch thô từ hai file `fraudTrain.csv` và `fraudTest.csv`. Sau đó, dữ liệu được đưa vào Spark DataFrame để thực hiện các thao tác kiểm tra, tổng hợp và phân tích.

Các bước chính trong quá trình xử lý bằng Spark gồm:

- Đọc dữ liệu giao dịch từ file CSV vào Spark DataFrame;
- Kiểm tra số lượng bản ghi, số lượng giao dịch hợp lệ và giao dịch gian lận;
- Tính tỷ lệ giao dịch gian lận trên toàn bộ dữ liệu;
- Tổng hợp số lượng giao dịch và tỷ lệ gian lận theo từng nhóm thuộc tính;
- Xuất các bảng kết quả ra thư mục báo cáo để phục vụ phân tích và trình bày.

Các phép phân tích bằng Spark chủ yếu dựa trên các thao tác `groupBy`, `agg`, `count`, `sum` và tính toán tỷ lệ gian lận. Đây là các thao tác phổ biến trong xử lý dữ liệu lớn, đặc biệt phù hợp với các bài toán cần thống kê dữ liệu theo nhiều chiều.

3.3 Các phân tích thực hiện bằng Spark

Trong đề tài, Spark được sử dụng để tạo ra một số bảng phân tích chính. Các bảng này giúp mô tả dữ liệu rõ hơn và làm nổi bật các yếu tố có thể liên quan đến hành vi gian

lặn.

Bảng 3: Các bảng phân tích được sinh bằng Spark

Bảng kết quả	Nội dung phân tích
t3.1_dataset_summary.csv	Tổng quan số lượng giao dịch và tỷ lệ gian lận
t3.2_fraud_by_category.csv	Gian lận theo loại giao dịch
t3.3_fraud_by_state.csv	Gian lận theo bang
t3.4_high_risk_merchants.csv	Các merchant có rủi ro gian lận cao
t3.5_fraud_by_hour.csv	Gian lận theo giờ giao dịch
t3.6_amount_by_target.csv	Thống kê số tiền giao dịch theo nhãn

Trước hết, bảng tổng quan dữ liệu cho biết số lượng giao dịch, số lượng giao dịch gian lận, số lượng giao dịch hợp lệ và tỷ lệ gian lận. Kết quả này giúp khẳng định đặc điểm mất cân bằng mạnh của bộ dữ liệu.

Phân tích theo **category** cho thấy tỷ lệ gian lận không đồng đều giữa các loại giao dịch. Một số nhóm giao dịch có tỷ lệ gian lận cao hơn các nhóm còn lại, cho thấy **category** là một thuộc tính có ý nghĩa trong bài toán phát hiện gian lận.

Phân tích theo **state** giúp quan sát sự khác biệt về tỷ lệ gian lận giữa các khu vực địa lý. Mặc dù không thể kết luận trực tiếp rằng một bang nhất định luôn có rủi ro cao hơn, kết quả này vẫn cung cấp thêm góc nhìn về phân bố gian lận theo vị trí.

Phân tích các merchant có rủi ro cao giúp nhận diện những merchant có tỷ lệ giao dịch gian lận đáng chú ý. Tuy nhiên, khi diễn giải kết quả này cần xét đến số lượng giao dịch của từng merchant. Một merchant có tỷ lệ gian lận cao nhưng số lượng giao dịch quá nhỏ có thể chưa đủ cơ sở để kết luận chắc chắn.

Phân tích theo giờ giao dịch giúp quan sát sự thay đổi của tỷ lệ gian lận trong ngày. Kết quả này hỗ trợ cho việc tạo các đặc trưng thời gian như **transaction_hour**, **transaction_dayofweek**, **transaction_month** và **weekend_flag** trong bước xây dựng đặc trưng.

Cuối cùng, thống kê số tiền giao dịch theo nhãn **is_fraud** giúp so sánh sự khác biệt về giá trị giao dịch giữa giao dịch hợp lệ và giao dịch gian lận. Điều này cho thấy biến **amt** là một đặc trưng quan trọng cần được giữ lại và chuẩn hóa trong dữ liệu đầu vào của mô hình.

3.4 Ý nghĩa của Spark đối với quá trình xây dựng mô hình

Mặc dù Spark không trực tiếp tạo ra dữ liệu đầu vào cuối cùng cho mô hình học máy, các kết quả phân tích từ Spark vẫn có ý nghĩa quan trọng đối với quá trình xây dựng mô hình. Spark giúp tổng hợp dữ liệu theo nhiều chiều khác nhau, từ đó cung cấp các *feature insights* hỗ trợ bước Feature Engineering.

Ví dụ, nếu kết quả phân tích bằng Spark cho thấy tỷ lệ gian lận khác nhau theo `category`, thì việc giữ lại và mã hóa thuộc tính `category` là hợp lý. Nếu gian lận thay đổi theo giờ giao dịch, thì các đặc trưng như `transaction_hour` và `weekend_flag` có thể giúp mô hình học được các mẫu hành vi theo thời gian. Tương tự, các phân tích theo `merchant`, vị trí và số tiền giao dịch cũng hỗ trợ quá trình lựa chọn và xây dựng đặc trưng.

Như vậy, Spark đóng vai trò như một nhánh phân tích dữ liệu lớn nhằm tạo insight cho quá trình xây dựng mô hình. Điều này giúp đề tài không chỉ dừng lại ở việc huấn luyện mô hình bằng scikit-learn, mà còn thể hiện được quy trình phân tích dữ liệu lớn trước khi đưa dữ liệu vào mô hình học máy.

3.5 So sánh Pandas và Spark

Trong đề tài, Pandas và Spark đều có thể được sử dụng cho các tác vụ phân tích dữ liệu. Tuy nhiên, hai công cụ này có đặc điểm và phạm vi phù hợp khác nhau.

Pandas phù hợp với dữ liệu có kích thước vừa và nhỏ, chạy trên một máy đơn và có cú pháp đơn giản, thuận tiện cho phân tích nhanh. Trong khi đó, Spark được thiết kế cho xử lý dữ liệu lớn, có khả năng xử lý song song và có thể mở rộng lên môi trường cluster khi dữ liệu tăng kích thước.

Bảng 4: So sánh Pandas và Spark trong xử lý dữ liệu

Tiêu chí	Pandas	Apache Spark
Môi trường xử lý	Một máy đơn	Phân tán, có thể mở rộng
Kiểu dữ liệu phù hợp	Nhỏ đến vừa	Lớn, nhiều bản ghi
Cách xử lý	In-memory trên một máy	Song song trên nhiều partition
Ưu điểm	Dễ dùng, linh hoạt	Khả năng mở rộng tốt
Hạn chế	Khó mở rộng khi dữ liệu lớn	Có overhead khởi tạo

Với bộ dữ liệu trong đề tài, Pandas vẫn có thể xử lý được trên máy cá nhân. Tuy nhiên, việc sử dụng Spark giúp mô phỏng quy trình xử lý dữ liệu lớn tốt hơn và phù hợp với định hướng của học phần. Trong trường hợp dữ liệu tăng lên nhiều lần hoặc cần triển khai trên hệ thống phân tán, Spark sẽ có lợi thế rõ ràng hơn về khả năng mở rộng.

4 Xây dựng mô hình phát hiện giao dịch gian lận

4.1 Phát biểu bài toán

Bài toán phát hiện giao dịch gian lận trong đề tài được xây dựng dưới dạng bài toán phân loại nhị phân. Với mỗi giao dịch đầu vào, mô hình cần dự đoán giao dịch đó là giao dịch hợp lệ hay giao dịch gian lận.

Biến mục tiêu của bài toán là `is_fraud`, trong đó:

- 0: giao dịch hợp lệ;
- 1: giao dịch gian lận.

Sau quá trình tiền xử lý và xây dựng đặc trưng, mỗi giao dịch được biểu diễn dưới dạng một vector đặc trưng số. Các đặc trưng này bao gồm thông tin về số tiền giao dịch, thời gian giao dịch, thông tin khách hàng, thông tin merchant, vị trí địa lý và các đặc trưng được tạo mới như tuổi khách hàng, khoảng cách giữa khách hàng và merchant.

Mục tiêu của mô hình là học từ dữ liệu giao dịch trong quá khứ để dự đoán nhãn `is_fraud` cho các giao dịch mới. Do số lượng giao dịch gian lận rất nhỏ so với giao dịch hợp lệ, bài toán này cần được xử lý như một bài toán dữ liệu mất cân bằng.

4.2 Xử lý mất cân bằng dữ liệu

Trong tập huấn luyện, giao dịch gian lận chỉ chiếm khoảng 0.5789% tổng số giao dịch. Điều này có nghĩa là phần lớn dữ liệu thuộc về lớp giao dịch hợp lệ. Nếu huấn luyện trực tiếp trên dữ liệu gốc, mô hình có thể thiên về lớp đa số và dự đoán hầu hết giao dịch là hợp lệ. Khi đó, Accuracy có thể cao nhưng khả năng phát hiện giao dịch gian lận lại thấp.

Để giảm ảnh hưởng của mất cân bằng dữ liệu, đề tài sử dụng chiến lược lấy mẫu lại trên tập huấn luyện. Cụ thể, toàn bộ các giao dịch gian lận được giữ lại, trong khi các giao dịch hợp lệ được lấy mẫu theo tỷ lệ nhất định. Trong đề tài này, tỷ lệ lấy mẫu được sử dụng là 5:1, tức là số lượng giao dịch hợp lệ trong tập huấn luyện sau lấy mẫu gấp 5 lần số lượng giao dịch gian lận.

Chiến lược này giúp mô hình quan sát được nhiều mẫu gian lận hơn trong quá trình học, từ đó cải thiện khả năng nhận diện lớp thiểu số. Tuy nhiên, việc lấy mẫu chỉ được thực hiện trên tập huấn luyện. Tập kiểm thử vẫn được giữ nguyên theo phân bố ban đầu để kết quả đánh giá phản ánh đúng đặc điểm dữ liệu thực tế.

Có thể tóm tắt chiến lược xử lý mất cân bằng như sau:

- Giữ lại toàn bộ giao dịch gian lận trong tập huấn luyện;
- Lấy mẫu một phần giao dịch hợp lệ theo tỷ lệ 5:1 so với số lượng giao dịch gian lận;
- Huấn luyện mô hình trên tập dữ liệu đã được lấy mẫu;
- Đánh giá mô hình trên toàn bộ tập kiểm thử ban đầu.

Cách làm này giúp mô hình học tốt hơn đặc điểm của giao dịch gian lận mà vẫn đảm bảo kết quả đánh giá cuối cùng không bị làm sai lệch bởi quá trình lấy mẫu.

4.3 Các mô hình sử dụng

Trong đề tài, ba mô hình học máy được sử dụng để thực nghiệm và so sánh gồm Logistic Regression, Random Forest và HistGradientBoosting. Việc lựa chọn nhiều mô hình giúp đánh giá hiệu quả của các nhóm phương pháp khác nhau, từ mô hình tuyến tính đơn giản đến các mô hình ensemble phức tạp hơn.

Bảng 5: Các mô hình được sử dụng trong đề tài

Mô hình	Vai trò trong đề tài
Logistic Regression	Mô hình tuyến tính, được sử dụng làm baseline để so sánh với các mô hình phức tạp hơn.
Random Forest	Mô hình ensemble dựa trên nhiều cây quyết định, có khả năng học các quan hệ phi tuyến trong dữ liệu.
HistGradientBoosting	Mô hình boosting dựa trên cây quyết định, có khả năng học tuần tự để cải thiện sai số của các mô hình trước đó.

4.3.1 Logistic Regression

Logistic Regression là mô hình tuyến tính thường được sử dụng cho các bài toán phân loại nhị phân. Mô hình này có ưu điểm là đơn giản, tốc độ huấn luyện nhanh và dễ diễn giải. Trong đề tài, Logistic Regression được sử dụng như một mô hình baseline nhằm đánh giá mức hiệu quả ban đầu của pipeline.

Tuy nhiên, do bản chất tuyến tính, Logistic Regression có thể gặp hạn chế khi dữ liệu có quan hệ phức tạp hoặc phi tuyến giữa các đặc trưng. Vì vậy, kết quả của Logistic Regression chủ yếu được dùng làm mốc so sánh với các mô hình mạnh hơn.

4.3.2 Random Forest

Random Forest là mô hình ensemble gồm nhiều cây quyết định. Mỗi cây được huấn luyện trên một phần dữ liệu và một phần đặc trưng khác nhau, sau đó kết quả dự đoán của các cây được tổng hợp để đưa ra dự đoán cuối cùng.

Ưu điểm của Random Forest là có khả năng học các quan hệ phi tuyến và giảm hiện tượng overfitting so với một cây quyết định đơn lẻ. Trong bài toán phát hiện gian lận, Random Forest có thể khai thác tốt các tương tác giữa nhiều đặc trưng như số tiền giao dịch, thời gian, loại giao dịch và vị trí địa lý.

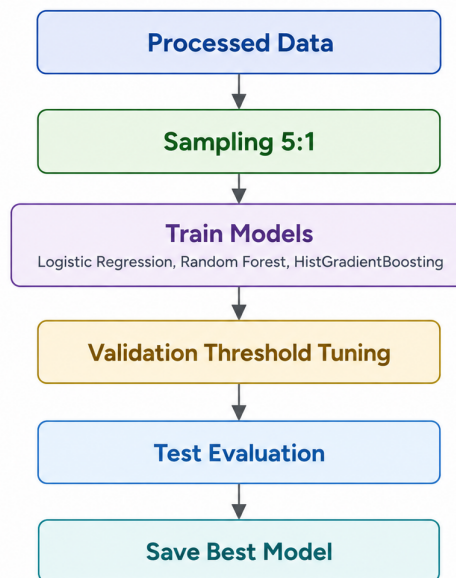
4.3.3 HistGradientBoosting

HistGradientBoosting là mô hình boosting dựa trên cây quyết định. Khác với Random Forest, các cây trong boosting được xây dựng tuần tự, trong đó mô hình sau tập trung cải thiện các lỗi của mô hình trước. Cách học này giúp mô hình có khả năng biểu diễn mạnh và thường đạt hiệu quả cao trong các bài toán dữ liệu dạng bảng.

Trong đề tài, HistGradientBoosting là mô hình quan trọng nhất trong nhóm thực nghiệm, vì mô hình này cho kết quả tốt nhất sau quá trình đánh giá. Mô hình có khả năng cân bằng tốt hơn giữa Precision và Recall, đồng thời đạt kết quả cao trên các độ đo phù hợp với dữ liệu mất cân bằng như F1-score và PR-AUC.

4.4 Quy trình huấn luyện mô hình

Quy trình huấn luyện mô hình được thực hiện sau khi dữ liệu đã được tiền xử lý và xây dựng đặc trưng. Dữ liệu đầu vào của mô hình gồm các đặc trưng số đã được mã hóa, chuẩn hóa và biến mục tiêu `is_fraud`. Toàn bộ quy trình huấn luyện được tóm tắt trong Hình 6.



Hình 6: Quy trình huấn luyện và đánh giá mô hình

Quy trình huấn luyện có thể mô tả qua các bước sau:

- Tách đặc trưng đầu vào và biến mục tiêu `is_fraud`;
- Áp dụng chiến lược lấy mẫu trên tập huấn luyện để xử lý mất cân bằng dữ liệu;
- Huấn luyện lần lượt các mô hình Logistic Regression, Random Forest và HistGradientBoosting;
- Sinh xác suất dự đoán gian lận cho từng giao dịch;

- Điều chỉnh ngưỡng dự đoán trên tập validation;
- Đánh giá mô hình trên toàn bộ tập kiểm thử.

Trong quá trình huấn luyện, các bước xử lý như mã hóa, chuẩn hóa và lấy mẫu được thực hiện cẩn thận nhằm tránh rò rỉ dữ liệu. Các thông tin dùng để biến đổi dữ liệu được học từ tập huấn luyện, sau đó áp dụng lại cho tập kiểm thử. Tập kiểm thử không được sử dụng để lựa chọn tham số hoặc điều chỉnh ngưỡng, mà chỉ được dùng để đánh giá kết quả cuối cùng.

4.5 Điều chỉnh ngưỡng dự đoán (Threshold Tuning)

Các mô hình phân loại thường trả về xác suất một giao dịch thuộc lớp gian lận. Theo mặc định, nếu xác suất lớn hơn hoặc bằng 0.5 thì giao dịch được dự đoán là gian lận, ngược lại được dự đoán là hợp lệ. Tuy nhiên, trong bài toán dữ liệu mất cân bằng, ngưỡng 0.5 không phải lúc nào cũng là lựa chọn phù hợp.

Nếu ngưỡng dự đoán quá thấp, mô hình có thể phát hiện được nhiều giao dịch gian lận hơn nhưng đồng thời tạo ra nhiều cảnh báo nhầm. Khi đó, Recall có thể tăng nhưng Precision có thể giảm. Ngược lại, nếu ngưỡng dự đoán quá cao, mô hình sẽ cảnh báo ít hơn, Precision có thể tăng nhưng có nguy cơ bỏ sót nhiều giao dịch gian lận, làm Recall giảm.

Do đó, đề tài thực hiện điều chỉnh ngưỡng dự đoán dựa trên F1-score trên tập validation. F1-score được sử dụng vì đây là độ đo cân bằng giữa Precision và Recall, phù hợp với bài toán cần vừa phát hiện được nhiều giao dịch gian lận, vừa hạn chế số lượng cảnh báo sai.

Quá trình điều chỉnh ngưỡng có thể hiểu như sau:

- Mô hình sinh xác suất gian lận cho từng giao dịch;
- Thử nhiều giá trị ngưỡng khác nhau trên tập validation;
- Tính Precision, Recall và F1-score ứng với từng ngưỡng;
- Chọn ngưỡng cho F1-score tốt nhất;
- Sử dụng ngưỡng đã chọn để đánh giá trên tập kiểm thử.

Việc điều chỉnh ngưỡng giúp mô hình phù hợp hơn với mục tiêu của bài toán phát hiện gian lận, thay vì phụ thuộc hoàn toàn vào ngưỡng mặc định 0.5.

5 Kết quả thực nghiệm và đánh giá

5.1 Các độ đo đánh giá

Do dữ liệu giao dịch gian lận có mức độ mất cân bằng rất cao, việc đánh giá mô hình chỉ bằng Accuracy là chưa đủ. Nếu mô hình dự đoán phần lớn giao dịch là hợp lệ, Accuracy vẫn có thể đạt giá trị cao nhưng mô hình lại không phát hiện tốt các giao dịch gian lận. Vì vậy, trong đề tài này, các độ đo như Precision, Recall, F1-score, ROC-AUC và PR-AUC được sử dụng để đánh giá mô hình một cách phù hợp hơn.

Các độ đo được sử dụng có ý nghĩa như sau:

- **Accuracy**: tỷ lệ dự đoán đúng trên toàn bộ tập dữ liệu;
- **Precision**: trong các giao dịch được mô hình dự đoán là gian lận, tỷ lệ giao dịch thật sự là gian lận;
- **Recall**: trong toàn bộ giao dịch gian lận thật, tỷ lệ giao dịch được mô hình phát hiện đúng;
- **F1-score**: độ đo cân bằng giữa Precision và Recall;
- **ROC-AUC**: đo khả năng phân biệt hai lớp trên nhiều ngưỡng dự đoán khác nhau;
- **PR-AUC**: đo diện tích dưới đường cong Precision-Recall, đặc biệt có ý nghĩa trong bài toán dữ liệu mất cân bằng.

Trong bài toán phát hiện gian lận, Recall là một độ đo quan trọng vì nó phản ánh khả năng phát hiện các giao dịch gian lận. Nếu Recall thấp, nhiều giao dịch gian lận sẽ bị bỏ sót. Tuy nhiên, Precision cũng cần được quan tâm để hạn chế số lượng cảnh báo nhầm. Do đó, F1-score và PR-AUC được sử dụng để đánh giá sự cân bằng giữa hai yếu tố này.

5.2 Kết quả so sánh các mô hình

Sau khi huấn luyện và điều chỉnh ngưỡng dự đoán, các mô hình được đánh giá trên tập kiểm thử. Bảng dưới đây trình bày kết quả so sánh giữa ba mô hình Logistic Regression, Random Forest và HistGradientBoosting.

Bảng 6: Kết quả đánh giá các mô hình

Mô hình	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression tuned	0.1577	0.6699	0.2553	0.9055	0.1189
Random Forest	0.1802	0.9147	0.3011	0.9899	0.7441
HistGradientBoosting tuned	0.3557	0.9245	0.5137	0.9976	0.8701

Từ bảng kết quả có thể thấy Logistic Regression là mô hình baseline với kết quả thấp nhất trong ba mô hình. Mặc dù mô hình này có ưu điểm là đơn giản và dễ huấn luyện,

khả năng phát hiện giao dịch gian lận còn hạn chế do dữ liệu có quan hệ phức tạp và mất cân bằng mạnh.

Random Forest cải thiện rõ rệt so với Logistic Regression, đặc biệt ở Recall và PR-AUC. Điều này cho thấy mô hình cây quyết định dạng ensemble có khả năng học tốt hơn các quan hệ phi tuyến trong dữ liệu. Tuy nhiên, Precision của Random Forest vẫn còn thấp, dẫn đến F1-score chưa cao.

HistGradientBoosting là mô hình đạt kết quả tốt nhất trong các mô hình được thử nghiệm. Mô hình này đạt Recall 0.9245, F1-score 0.5137 và PR-AUC 0.8701. Đây là các kết quả quan trọng đối với bài toán phát hiện gian lận, vì chúng cho thấy mô hình không chỉ phát hiện được phần lớn giao dịch gian lận mà còn có khả năng cân bằng tốt hơn giữa Precision và Recall.

5.3 Phân tích mô hình tốt nhất

Mô hình tốt nhất trong đề tài là HistGradientBoosting sau khi điều chỉnh ngưỡng theo F1-score. Ngưỡng dự đoán tốt nhất được lựa chọn là 0.80. Điều này có nghĩa là một giao dịch sẽ được dự đoán là gian lận nếu xác suất gian lận do mô hình sinh ra lớn hơn hoặc bằng 0.80.

Kết quả của mô hình HistGradientBoosting tuned được tóm tắt như sau:

Bảng 7: Kết quả của mô hình HistGradientBoosting tuned

Độ đo	Giá trị
Threshold	0.80
Precision	0.3557
Recall	0.9245
F1-score	0.5137
ROC-AUC	0.9976
PR-AUC	0.8701

Recall đạt 0.9245 cho thấy mô hình có khả năng phát hiện khoảng 92.45% số giao dịch gian lận trong tập kiểm thử. Đây là kết quả quan trọng vì trong bài toán fraud detection, việc bỏ sót giao dịch gian lận có thể gây thiệt hại lớn.

Precision đạt 0.3557, nghĩa là trong các giao dịch bị mô hình gắn nhãn gian lận, có khoảng 35.57% giao dịch thực sự là gian lận. Mặc dù Precision chưa quá cao, kết quả này vẫn có ý nghĩa vì tỷ lệ gian lận ban đầu trong dữ liệu chỉ khoảng 0.5789%. Như vậy, mô hình đã giúp lọc ra một nhóm giao dịch có tỷ lệ gian lận cao hơn rất nhiều so với phân bố gốc của dữ liệu.

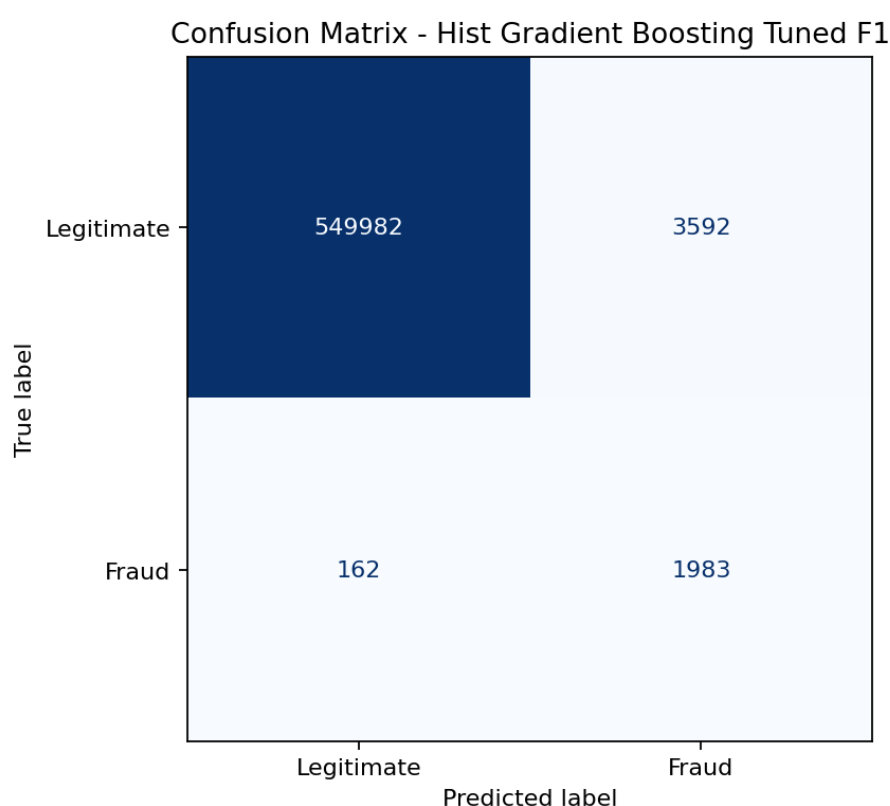
F1-score đạt 0.5137, cao hơn đáng kể so với các mô hình còn lại. Điều này cho thấy HistGradientBoosting cân bằng tốt hơn giữa khả năng phát hiện gian lận và hạn chế cảnh

báo nhầm. PR-AUC đạt 0.8701 cũng là một kết quả đáng chú ý, vì PR-AUC là độ đo quan trọng trong các bài toán có lớp dương rất hiếm như phát hiện gian lận.

5.4 Trực quan hóa kết quả đánh giá

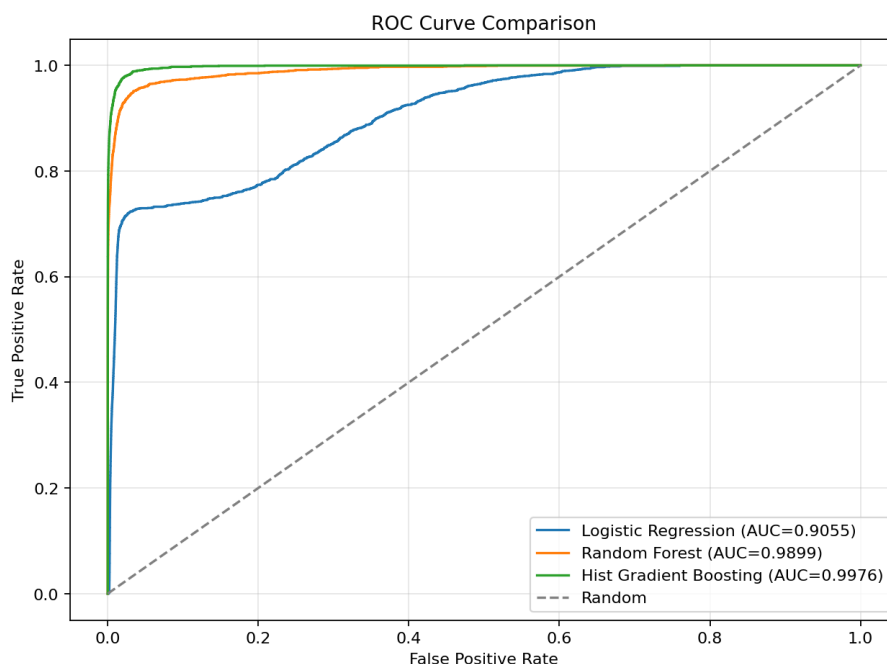
Bên cạnh các bảng số liệu, đề tài sử dụng một số biểu đồ để trực quan hóa kết quả đánh giá mô hình. Các biểu đồ này giúp quan sát rõ hơn hiệu quả phân loại của mô hình, đặc biệt là mô hình HistGradientBoosting.

Confusion matrix được sử dụng để quan sát số lượng giao dịch được phân loại đúng và sai ở từng lớp. Trong bài toán phát hiện gian lận, số lượng false negative là yếu tố cần quan tâm vì đây là các giao dịch gian lận thật nhưng bị mô hình dự đoán là hợp lệ.



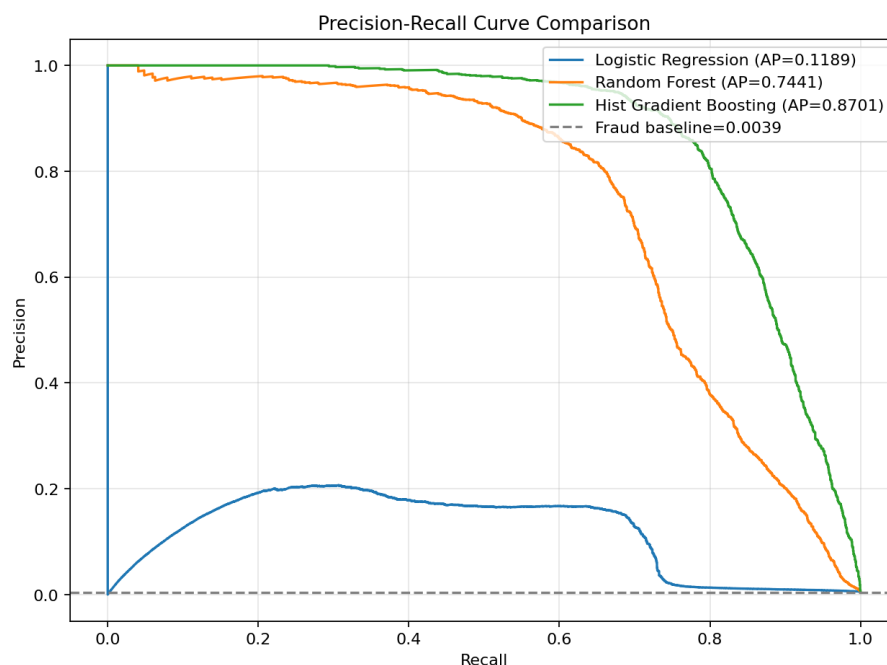
Hình 7: Confusion matrix của mô hình tốt nhất

Đường cong ROC thể hiện khả năng phân biệt giữa giao dịch hợp lệ và giao dịch gian lận trên nhiều ngưỡng dự đoán khác nhau. Mô hình có ROC-AUC càng cao thì khả năng phân biệt hai lớp càng tốt.



Hình 8: Đường cong ROC của các mô hình

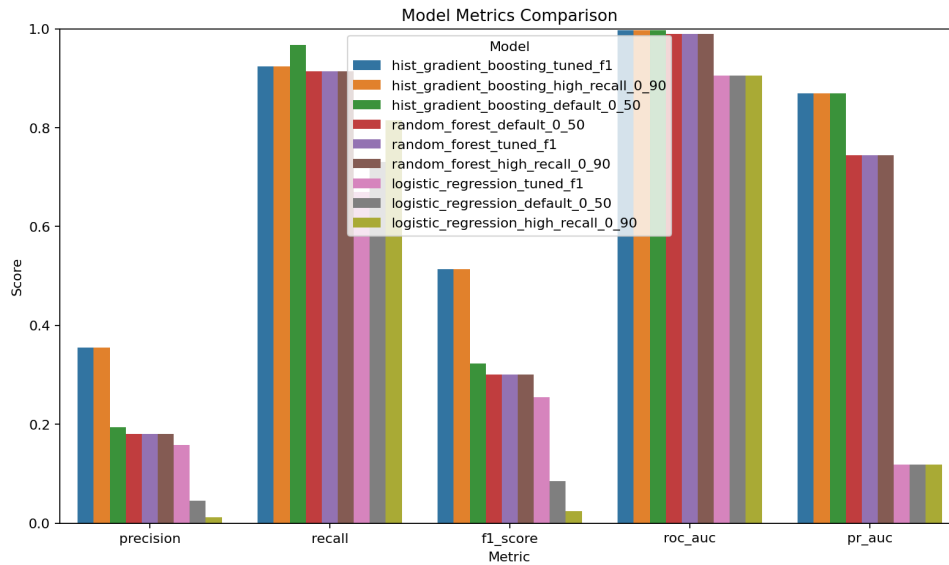
Đường cong Precision-Recall thể hiện mối quan hệ giữa Precision và Recall khi thay đổi ngưỡng dự đoán. Với dữ liệu mất cân bằng, Precision-Recall curve có ý nghĩa quan trọng hơn so với việc chỉ quan sát Accuracy.



Hình 9: Đường cong Precision-Recall của các mô hình

Ngoài ra, biểu đồ so sánh metric giữa các mô hình giúp quan sát trực quan sự khác biệt về Precision, Recall, F1-score, ROC-AUC và PR-AUC. Kết quả cho thấy

HistGradientBoosting vượt trội hơn các mô hình còn lại trên hầu hết các độ đo quan trọng.



Hình 10: So sánh các độ đo đánh giá giữa các mô hình

5.5 Nhận xét

Kết quả thực nghiệm cho thấy việc chỉ sử dụng một mô hình tuyến tính đơn giản như Logistic Regression là chưa đủ hiệu quả cho bài toán phát hiện giao dịch gian lận. Random Forest cải thiện đáng kể khả năng phát hiện gian lận, nhưng vẫn chưa đạt sự cân bằng tốt giữa Precision và Recall.

Kết quả cho thấy HistGradientBoosting tuned là mô hình hiệu quả nhất trong đề tài. Mặc dù Precision chưa đạt mức quá cao, điều này vẫn hợp lý trong bối cảnh tỷ lệ gian lận gốc rất thấp. Với Recall đạt 0.9245, F1-score đạt 0.5137 và PR-AUC đạt 0.8701, mô hình cho thấy khả năng phát hiện phần lớn giao dịch gian lận trong điều kiện dữ liệu mất cân bằng mạnh. Đây là cơ sở để lựa chọn HistGradientBoosting làm mô hình chính cho hệ thống phát hiện giao dịch gian lận.

6 Kết luận và hướng phát triển

6.1 Kết luận

Trong bài tập lớn này, đề tài đã xây dựng một pipeline phát hiện giao dịch gian lận sử dụng phân tích dữ liệu lớn và học máy. Pipeline bao gồm các bước chính như mô tả và phân tích dữ liệu, tiền xử lý, xây dựng đặc trưng, xử lý dữ liệu lớn bằng Apache Spark, huấn luyện mô hình và đánh giá kết quả.

Trong quá trình thực hiện, Apache Spark được sử dụng để xử lý và tổng hợp dữ liệu theo nhiều chiều như loại giao dịch, bang, merchant, giờ giao dịch và số tiền giao dịch. Các kết quả phân tích này hỗ trợ quá trình hiểu dữ liệu và xây dựng đặc trưng cho mô hình học máy.

Về phần mô hình, đề tài đã huấn luyện và so sánh ba mô hình gồm Logistic Regression, Random Forest và HistGradientBoosting. Kết quả thực nghiệm cho thấy HistGradientBoosting là mô hình tốt nhất, đạt Recall 0.9245, F1-score 0.5137 và PR-AUC 0.8701 sau khi điều chỉnh ngưỡng dự đoán. Điều này cho thấy mô hình có khả năng phát hiện phần lớn giao dịch gian lận trong điều kiện dữ liệu mất cân bằng mạnh.

6.2 Hạn chế của đề tài

Mặc dù pipeline đã đạt được kết quả khả quan, đề tài vẫn còn một số hạn chế:

- Đề tài mới dừng lại ở hướng xử lý theo lô trên dữ liệu có sẵn, chưa triển khai phát hiện gian lận theo thời gian thực;
- Apache Spark chủ yếu được sử dụng cho các tác vụ phân tích và tổng hợp dữ liệu, chưa được tích hợp trực tiếp vào toàn bộ pipeline huấn luyện hoặc scoring mô hình;
- Các mô hình được thử nghiệm còn giới hạn ở Logistic Regression, Random Forest và HistGradientBoosting, chưa mở rộng sang các mô hình như XGBoost, LightGBM hoặc CatBoost;
- Precision của mô hình tốt nhất chưa quá cao, nên hệ thống vẫn có thể tạo ra một số lượng cảnh báo nhầm;
- Đề tài chưa tập trung sâu vào khả năng giải thích mô hình, trong khi đây là yếu tố quan trọng đối với các bài toán trong lĩnh vực tài chính.

6.3 Hướng phát triển

Trong tương lai, đề tài có thể được phát triển theo một số hướng sau:

- Thử nghiệm thêm các mô hình mạnh hơn cho dữ liệu dạng bảng như XGBoost, LightGBM hoặc CatBoost;

- Thực hiện hyperparameter tuning sâu hơn để cải thiện Precision, Recall, F1-score và PR-AUC;
- Áp dụng cost-sensitive learning để tối ưu mô hình theo chi phí thực tế của false positive và false negative;
- Xây dựng batch scoring pipeline để tự động xử lý giao dịch mới và sinh cảnh báo rủi ro;
- Mở rộng sang xử lý thời gian thực bằng Spark Streaming hoặc Kafka;
- Tích hợp các phương pháp giải thích mô hình như SHAP để phân tích lý do một giao dịch bị đánh dấu là gian lận;
- Xây dựng dashboard trực quan để theo dõi tỷ lệ gian lận, số lượng cảnh báo và hiệu quả mô hình theo thời gian.

Tóm lại, đề tài đã xây dựng được một pipeline tương đối hoàn chỉnh cho bài toán phát hiện giao dịch gian lận, từ dữ liệu thô đến phân tích, xử lý, huấn luyện và đánh giá mô hình. Thông qua bài tập lớn này, em có cơ hội củng cố kiến thức về xử lý dữ liệu lớn, tiền xử lý dữ liệu, xây dựng đặc trưng, xử lý mất cân bằng dữ liệu và đánh giá mô hình học máy trong một bài toán thực tế.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy cô và các bạn trong lớp là những người đã giúp đỡ, đồng hành cùng em trong suốt quá trình học tập. Sự động viên và đóng góp của mọi người là nguồn động lực lớn để nhóm có thể hoàn thành bài tập này. Bài báo cáo chắc chắn vẫn còn nhiều thiếu sót em rất mong nhận được sự góp ý từ thầy cô để em có thể tiếp tục phát triển bài tập này nói riêng và phát triển bản thân mình trong các dự án học thuật tương lai nói chung.

Em xin kính chúc thầy cô nhiều sức khỏe, công tác tốt và luôn thành công trong hành trình nghiên cứu và giảng dạy cao quý của mình!

NGUỒN DỮ LIỆU VÀ TÀI LIỆU THAM KHẢO

- [1] Kaggle. *Credit Card Transactions Fraud Detection Dataset*. Available at: <https://www.kaggle.com/datasets/kartik2112/fraud-detection>.
- [2] M. Zaharia et al. *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM, 2016. Available at: <https://doi.org/10.1145/2934664>.
- [3] L. Breiman. *Random Forests*. Machine Learning, 2001. Available at: <https://doi.org/10.1023/A:1010933404324>.
- [4] J. H. Friedman. *Greedy Function Approximation: A Gradient Boosting Machine*. The Annals of Statistics, 2001. Available at: <https://doi.org/10.1214/aos/1013203451>.
- [5] H. He and E. A. Garcia. *Learning from Imbalanced Data*. IEEE Transactions on Knowledge and Data Engineering, 2009. Available at: <https://doi.org/10.1109/TKDE.2008.239>.
- [6] J. Davis and M. Goadrich. *The Relationship Between Precision-Recall and ROC Curves*. Proceedings of the 23rd International Conference on Machine Learning, 2006. Available at: <https://doi.org/10.1145/1143844.1143874>.
- [7] T. Saito and M. Rehmsmeier. *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLOS ONE, 2015. Available at: <https://doi.org/10.1371/journal.pone.0118432>.
- [8] Tài liệu và slide học phần: các nội dung về đặc trưng dữ liệu lớn 5V, Apache Spark, xử lý dữ liệu lớn, học máy và đánh giá mô hình.